

ui.label 全面详细阐述

在 NiceGUI 框架中，`ui.label` 是用于显示文本内容的基础 UI 组件，具备灵活的文本绑定、样式定制、事件处理等能力，适用于各类文本展示场景。以下从核心定义、基础使用、高级特性、属性与方法等维度进行全面解析。

一、核心定义与作用

`ui.label` 的核心功能是**显示文本内容**，作为 UI 界面中最基础的文本展示组件，它支持静态文本渲染、动态内容绑定、样式自定义等核心能力，且可与其他组件联动实现交互逻辑。其本质是对 HTML 文本元素的封装，结合 Vue 的响应式特性，实现前后端数据同步与界面更新。

二、基础使用

1. 最简示例

通过一行代码即可创建一个显示静态文本的标签，适用于无需动态更新的场景：

```
from nicegui import ui

# 创建静态文本标签
ui.label('some label')

ui.run()
```

运行后将在界面上直接显示文本 “some label”。

2. 动态文本与样式联动

通过重写 `_handle_text_change` 方法，可实现文本内容与组件样式的联动，适用于状态提示等场景（如 “成功 / 失败” 状态切换）：

```
from nicegui import ui

# 自定义标签类，重写文本变化处理方法
class StatusLabel(ui.label):
    def _handle_text_change(self, text: str) -> None:
        super()._handle_text_change(text) # 调用父类方法保持基础文本渲染
        # 根据文本内容切换样式类
        if text == 'ok':
            self.classes(replace='text-positive') # 成功状态：绿色文本
        else:
            self.classes(replace='text-negative') # 失败状态：红色文本

# 定义响应式数据模型
model = {'status': 'error'}

# 创建自定义标签并绑定数据（从model的status字段读取文本）
StatusLabel().bind_text_from(model, 'status')

# 开关组件，切换status值（联动标签文本与样式）
ui.switch(on_change=lambda e: model.update(status='ok' if e.value else 'error'))
```

`ui.run()`

- 初始状态: model 的 status 为 “error”，标签显示红色文本 “error”；
- 切换开关: 开关开启时，status 变为 “ok”，标签自动切换为绿色文本 “ok”；
- 核心逻辑: `_handle_text_change` 方法在文本更新时触发，通过 `classes(replace=...)` 替换样式类。

三、核心属性

`ui.label` 继承自 NiceGUI 的基础 `Element` 类，拥有以下核心属性（部分为 `BindableProperty`，支持响应式绑定）：

属性名	类型	说明
<code>text</code>	<code>BindableProperty</code>	标签的文本内容，支持双向绑定，值更新时界面自动刷新
<code>classes</code>	<code>Classes[Self]</code>	组件的 HTML 类（支持 Tailwind/Quasar 样式类），用于定制外观
<code>client</code>	<code>Client</code>	组件所属的客户端实例，用于前后端通信关联
<code>html_id</code>	<code>str</code>	HTML DOM 中的元素 ID（2.16.0 版本新增），用于直接操作 DOM 元素
<code>is_deleted</code>	<code>bool</code>	组件是否已被删除，只读属性
<code>is_ignoring_events</code>	<code>bool</code>	组件是否正在忽略事件，只读属性
<code>parent_slot</code>	<code>Slot None</code>	组件所属的父插槽（可设置），用于复杂布局中的插槽管理
<code>props</code>	<code>Props[Self]</code>	组件的 HTML 属性（Quasar props），用于扩展组件特性
<code>style</code>	<code>Style[Self]</code>	组件的内联 CSS 样式，用于精细化外观控制
<code>visible</code>	<code>BindableProperty</code>	组件的可见性，支持响应式绑定，值为 <code>True</code> 时显示， <code>False</code> 时隐藏

属性使用示例

```
# 定制样式与属性
label = ui.label('定制标签')
label.classes('text-xl font-bold') # 字体大小xl、加粗
label.style('color: #2c3e50; margin: 10px 0;') # 文本颜色、外边距
label.html_id = 'custom-label' # 设置DOM ID
label.visible = True # 显示组件
```

四、核心方法

`ui.label` 提供了丰富的方法用于文本操作、绑定、样式控制、事件处理等，以下是常用核心方法分类解析：

1. 文本操作方法

方法名	参数说明	功能描述
<code>set_text</code>	<code>text: str</code> : 新文本内容	直接设置标签文本，覆盖原有内容
<code>bind_text</code>	目标对象、目标属性名、正向 / 反向转换函数等	双向绑定文本：组件文本与目标对象属性相互同步
<code>bind_text_from</code>	目标对象、目标属性名、反向转换函数等	单向绑定（从目标到组件）：目标对象属性更新时，组件文本自动同步
<code>bind_text_to</code>	目标对象、目标属性名、正向转换函数等	单向绑定（从组件到目标）：组件文本更新时，目标对象属性自动同步

文本绑定示例

```
from nicegui import ui

# 双向绑定：标签与输入框同步文本
text_model = {'content': '初始文本'}
ui.label().bind_text(text_model, 'content') # 标签文本绑定到model
ui.input().bind_value(text_model, 'content') # 输入框值绑定到model

# 单向绑定（从目标到组件）：标签文本跟随变量更新
status = {'code': 'normal'}
ui.label().bind_text_from(status, 'code', backward=lambda x: f'状态: {x}') # 文本格式化

# 直接设置文本
label = ui.label('旧文本')
label.set_text('新文本') # 覆盖为“新文本”

ui.run()
```

2. 可见性绑定方法

用于控制组件的显示 / 隐藏，支持响应式绑定：

方法名	核心参数	功能描述
<code>set_visibility</code>	<code>visible: bool</code> : 是否可见	直接设置组件可见性
<code>bind_visibility</code>	目标对象、目标属性名、匹配值等	双向绑定可见性：组件与目标对象属性相互同步
<code>bind_visibility_from</code>	目标对象、目标属性名、匹配值等	单向绑定（从目标到组件）：目标属性满足条件时显示组件
<code>bind_visibility_to</code>	目标对象、目标属性名、转换函数等	单向绑定（从组件到目标）：组件可见性同步到目标对象属性

可见性绑定示例

```
from nicegui import ui

model = {'show_label': True, 'role': 'admin'}

# 直接设置可见性
label1 = ui.label('固定可见标签')
label1.set_visibility(True)

# 双向绑定：开关控制标签显示
ui.switch('显示标签', value=True).bind_value(model, 'show_label')
ui.label('动态可见标签').bind_visibility(model, 'show_label')

# 条件绑定：仅当role为admin时显示
ui.label('管理员专属标签').bind_visibility_from(model, 'role', value='admin')

ui.run()
```

3. 样式与外观定制方法

方法名	核心参数	功能描述
classes	add/remove/replace/toggle：样式类操作	新增、删除、替换或切换 HTML 样式类（支持 Tailwind/Quasar 类）
default_classes	同 classes，但作用于类级（所有实例共享）	定义该类所有标签的默认样式类（需在实例化前调用）
style	内联 CSS 样式字符串	设置组件内联样式，优先级高于 classes
default_style	同 style，作用于类级	定义该类所有标签的默认内联样式
tooltip	text: str：提示文本	为组件添加鼠标悬浮提示

样式定制示例

```
from nicegui import ui

# 类级默认样式：所有CustomLabel实例默认加粗、灰色文本
class CustomLabel(ui.label):
    default_classes(replace='font-bold text-gray-600')
    default_style(add='font-size: 16px;')

# 实例级样式修改
label1 = CustomLabel('类默认样式')
label2 = CustomLabel('实例自定义样式')
label2.classes(add='text-blue-500') # 新增蓝色文本
label2.style('margin-top: 10px;') # 新增上外边距
label2.tooltip('这是带提示的标签') # 悬浮提示
```

```
ui.run()
```

4. 事件与交互方法

方法名	核心参数	功能描述
<code>on</code>	事件类型、处理函数、JS 处理函数等	绑定事件（如 click、mousedown），支持 Python/JS 双端处理
<code>mark</code>	标记字符串（单个或多个）	为组件添加标记，用于测试查询或依赖管理
<code>update</code>	无参数	强制更新组件，同步后端状态到前端

事件绑定示例

```
from nicegui import ui

label = ui.label('点击我')
label.classes('cursor-pointer text-blue-500') # 鼠标悬浮变指针，蓝色文本

# 绑定点击事件（Python后端处理）
label.on('click', lambda e: label.set_text('已点击'))

# 绑定鼠标悬浮事件（JS前端处理，无需后端通信）
label.on('mouseover', js_handler='(e) => e.target.style.color = "red"')
label.on('mouseout', js_handler='(e) => e.target.style.color = "#3b82f6"')

ui.run()
```

5. 布局与容器相关方法

方法名	核心参数	功能描述
<code>add_slot</code>	插槽名、Vue 模板	为组件添加 Vue 插槽，用于复杂子元素布局（如自定义子组件插入）
<code>move</code>	目标容器、目标索引、目标插槽	将组件移动到其他容器或插槽中
<code>clear</code>	无参数	删除组件的所有子元素
<code>remove</code>	子元素实例或 ID	删除指定子元素
<code>delete</code>	无参数	删除组件自身及所有子元素

6. 资源与依赖管理方法

方法名	核心参数	功能描述
<code>add_resource</code>	资源路径（文件夹 / 文件）	为组件添加静态资源（如 CSS/JS 文件）
<code>add_dynamic_resource</code>	资源名、生成函数	为组件添加动态资源（函数返回资源响应）
<code>get_computed_prop</code>	属性名、超时时间	异步获取组件的计算属性（需 <code>await</code> ）
<code>run_method</code>	方法名、参数、超时时间	调用客户端侧方法（如 Vue 组件方法），支持异步获取返回值

五、高级特性

1. 响应式绑定机制

`ui.label` 的 `text` 和 `visible` 属性均为可绑定属性（`BindableProperty`），结合 `bind_*` 系列方法，可实现：

- 数据驱动界面：当绑定的对象属性变化时，标签文本 / 可见性自动更新；
- 双向同步：通过 `bind_text` 实现标签文本与输入框、下拉框等组件的值双向同步；
- 数据转换：通过 `forward/backward` 参数对绑定数据进行格式化（如添加前缀、类型转换）。

2. 自定义标签类

通过继承 `ui.label` 并重写方法（如 `_handle_text_change`），可实现个性化逻辑：

- 文本格式化：自动为文本添加前缀 / 后缀（如 “[提示] 文本内容”）；
- 状态联动：根据文本内容切换样式、显示图标或触发其他组件行为；
- 权限控制：结合用户角色动态控制标签可见性或文本内容。

3. 与其他组件联动

`ui.label` 常作为“状态显示端”与交互组件（如开关、滑块、输入框）联动，例如：

- 开关控制状态文本（如上述动态样式示例）；
- 滑块控制文本大小（通过绑定滑块值到标签的 `style` 或 `classes`）；
- 输入框实时同步文本（通过 `bind_text` 双向绑定）。

六、版本兼容性说明

部分属性和方法存在版本限制，使用时需注意：

- `html_id`：2.16.0 版本新增；
- `default_classes` 的 `toggle` 参数：2.7.0 版本新增；
- `on` 方法支持同时指定 Python 和 JS 处理函数：2.18.0 版本更新；
- `strict` 参数（绑定方法中）：3.0.0 版本新增，用于校验目标对象的属性是否存在。

七、适用场景总结

1. 静态文本展示：如标题、描述、说明文字等；
2. 动态状态提示：如操作结果（成功 / 失败）、系统状态（在线 / 离线）等；
3. 数据同步显示：如实时展示传感器数据、用户输入内容等；
4. 交互反馈：结合事件绑定，显示点击、悬浮等交互后的反馈文本。

通过上述特性，`ui.Label` 成为 NiceGUI 中最基础且灵活的文本组件，既能满足简单的文本展示需求，也能通过高级特性实现复杂的响应式交互逻辑。